AI Structural-Break Detector for Statistical Arbitrage

Supervised ML Pipeline

Complete event-dataset, feature-engineering, training, calibration, explainability, evaluation, and inference specification

This document begins where the Econometric Relationship Engine ends: eligible divergence events are converted into a point-in-time supervised-learning dataset, transformed into versioned features, used to train and calibrate the classifier, and packaged as a reproducible inference artefact for the application.

Prediction objective

For each historical divergence event, estimate the probability that the spread will satisfy the pre-defined mean-reversion success rule. The ML model does not forecast exact stock prices, exact trade returns, or the final CANDIDATE / REVIEW / NO TRADE / ABSTAIN decision.

Pipeline boundary

The core v1 ML model uses quantitative point-in-time features from prices, econometric diagnostics, market state, security state, and liquidity. Structured NLP/event features are a Model B extension and are never required for the first functional ML system.

Specification date: 28 September 2026

Contents

1. Pipeline role, boundaries, and end-to-end workflow
2. Inputs received from the econometric and data layers
3. Divergence-event generation
4. Target construction and outcome resolution
5. Event de-duplication, censoring, and metadata
6. Feature engineering design principles
7. Core feature set: spread state
8. Core feature set: relationship quality
9. Core feature set: market context state
10. Core feature set: security state and liquidity
11. Structured event / NLP features for Model B
12. Supervised dataset schema and storage
13. Chronological partitions, embargo, and model-selection protocol
14. Preprocessing and training/inference data contracts
15. Baseline model: logistic regression
16. Primary model: XGBoost classifier
17. Hyperparameter tuning and model selection
18. Class imbalance, weighting, and sample dependence
19. Probability calibration
20. Model thresholding and probability buckets
21. SHAP explainability
22. Predictive evaluation and calibration diagnostics
23. ML ablations, robustness, and finance handoff
24. Artefacts, versioning, experiment tracking, and persistence
25. Inference workflow, UI integration, and failure handling
26. Automated tests, monitoring, build sequence, and acceptance criteria

Appendix A. Working configuration defaults

Appendix B. Formula and feature reference

References and course-source basis

Parameter discipline

Defaults in this document are working implementation choices. Hyperparameters, feature-family inclusion, calibration, and policy thresholds may be changed using training/validation evidence only. The 2024-2025 final test period remains untouched until the methodology is frozen.

1. Pipeline role, boundaries, and end-to-end workflow

1.1 Responsibility

The supervised ML pipeline turns discrete econometric divergence events into calibrated probabilities. It owns event/target construction, point-in-time feature creation, dataset partitioning, preprocessing, model training, probability calibration, SHAP explanations, predictive evaluation, artefact packaging, and inference. It does not own pair estimation, document retrieval, LLM reasoning, the final decision controller, or simulated execution.

1.2 End-to-end workflow

Step	Operation	Primary output
1	Receive ACTIVE pair state and daily frozen spread/z-score from the Relationship Engine.	Point-in-time candidate state.
2	Detect a fresh threshold crossing and open one divergence event.	event_id and entry metadata.
3	Freeze all predictors available at event time t0.	Versioned feature snapshot X_i.
4	Look forward only for target construction until convergence, stop, or horizon.	Binary target y_i plus outcome metadata.
5	Build the event-level supervised table and run leakage/data-contract checks.	Model-ready development dataset.
6	Split chronologically; tune inside training with forward folds.	Frozen model specification.
7	Fit the final training-period XGBoost model.	Uncalibrated model scores.
8	Fit Platt/sigmoid calibration on validation predictions.	Calibrated probability P(revert).
9	Evaluate on untouched final test data and compute SHAP explanations.	Out-of-sample metrics and explanations.
10	Persist model bundle and load it in the dashboard for current/historical inference.	Reproducible app inference.

Central information-set rule

For an event at time t0, every predictor must be measurable from information available at or before t0. Future observations may be used only to determine the target. This rule is enforced both in dataset construction and automated tests.

2. Inputs received from the econometric and data layers

2.1 Required upstream tables

Input	Required fields	Why ML needs it
pair_snapshots	pair_id, as_of_date, status, alpha, beta, coint statistic/p-value, half-life, correlations, beta drift	Relationship-quality predictors and eligibility.
daily pair state	date, pair_id, spread, z-score, spread mean/std, frozen snapshot_id	Event trigger and spread-state features.
prices	ticker, date, adjusted OHLCV	Returns, volatility, volume, dollar-volume, price-gap features.
market series	SPY / broad market, sector ETF, VIX	Market-context predictors.
pair registry	pair_id, y_ticker, x_ticker, sector ETF mapping	Stable identities and sector benchmark lookup.
Model B event extractions	event_id, type, affected leg, materiality, confidence, source metadata	Model B only; never required for Model A.

2.2 Fields that must never become predictors

- Future event outcome, exit date, exit z-score, time-to-convergence, stop flag, maximum adverse excursion, favourable excursion, trade return, or any other label-derived field.
- Pair ID, ticker symbols, company names, accession IDs, or other identifiers that let the classifier memorise specific securities rather than learn event structure.
- Any pair diagnostic fitted after the event timestamp, including a later hedge ratio or later cointegration result.
- Post-event SEC filings, later news explanations, or revised data that would not have been known at the event timestamp.

End-of-day convention

The prototype is an end-of-day research system. An event triggered using day t closing prices may use day t OHLCV and market data, because inference is assumed to occur after the close. The target begins at t+1.

3. Divergence-event generation

3.1 Unit of observation

The supervised-learning row is an event, not a trading day. A new row is created only when an ACTIVE pair newly crosses the configured divergence boundary. This matches the actual prediction task: score unusual dislocations, not normal market days.

$$E_i = (pair_i, t_i), \qquad |z_{i,t_i}| \geq z_{entry}, \quad |z_{i,t_i-1}| < z_{entry}$$

The baseline uses z_entry = 2.0. Requiring a crossing from inside to outside the band prevents a long multi-day divergence from generating one highly dependent training observation every day.

3.2 Event metadata frozen at entry

Field	Meaning
event_id	Stable unique identifier generated from pair_id + timestamp + label-policy version.
pair_id	Reference to the curated pair; not a model feature.
event_timestamp	End-of-day timestamp at which the crossing is observed.
entry_z	Signed z-score at entry.
divergence_sign	+1 if z > 0, -1 if z < 0.
snapshot_id	Exact econometric relationship snapshot used to compute the event.
feature_schema_version	Definition of every predictor expected for this event.
label_policy_version	Defines z_entry, convergence band, stop and horizon.
feature_cutoff_timestamp	Maximum timestamp any feature is allowed to use; normally event_timestamp.

3.3 Event eligibility gates

- Pair status must be ACTIVE at t0.
- Relationship snapshot must be valid and frozen before or at t0.
- Required quantitative features must pass data-quality checks.
- No unresolved prior event for the same pair.
- The event must have a full future label horizon inside its assigned dataset partition, otherwise it is excluded from supervised training/evaluation.

4. Target construction and outcome resolution

4.1 Binary target

For event i at t0, the target answers one question: did the spread successfully return to the configured equilibrium zone before reaching the adverse boundary and before the maximum horizon expired?

$$s_i = \text{sign}(z_{i,\,t_0}) \in \{-1,\ +1\}$$

$$\tau_i^{conv} = \min\{h \in \{1, \ldots, H\} : |z_{i,\,t_0+h}| \le z_{exit}\}$$

$$\tau_i^{stop} = \min\{h \in \{1, \ldots, H\} : s_i z_{i,\,t_0+h} \ge z_{stop}\}$$

$$y_i = \mathbf{1}\left[\tau_i^{conv} \le H \text{ and } \tau_i^{conv} < \tau_i^{stop}\right]$$

Baseline configuration: H = 10 trading days, z_exit = 0.5, and z_stop = 3.0. If convergence never occurs within H, the event is labelled failure even if it never touches the stop. This converts the financial research rule into an explicit supervised-learning target.

4.2 Outcome metadata retained beside the binary label

Stored field	Values / meaning	Model feature?
outcome_type	CONVERGED / STOPPED / TIMEOUT	No
resolution_date	First date at convergence/stop, otherwise horizon end	No
time_to_resolution	Trading days from t0 to resolution	No
max_adverse_z	Worst movement further in the initial divergence direction	No
max_favourable_z	Largest movement toward/through equilibrium	No
gross_pair_return / net_pair_return	Computed later by backtester	No

Why keep outcome metadata if it is not a feature?

It allows error analysis. For example, two y=0 failures may be economically different: one immediately hit the adverse stop while another merely timed out near equilibrium. The classifier is still trained on the binary target, but diagnostics can inspect these subtypes.

5. Event de-duplication, censoring, and metadata

5.1 One divergence episode -> one event

Once an event opens, no second event for the same pair is created until the first one resolves. After resolution, a new event requires the spread to return inside the entry band and subsequently cross outward again. This avoids repeated rows representing the same economic episode.

5.2 Partition censoring and embargo

Events whose future label window crosses a train/validation/test boundary are excluded from the earlier partition. This prevents a training label from using prices inside validation or test. The embargo length is at least the maximum 10-trading-day label horizon.

Case	Treatment
Event opens 5 days before validation starts	Exclude from training because its label could use validation-period prices.
Event opens during validation and resolves within validation	Valid validation row.
Event opens 3 days before final dataset end	Exclude from supervised evaluation because full H is unavailable.
Pair delists or data becomes unusable before resolution	Mark censored/invalid; exclude from classifier target unless a pre-specified failure rule covers the event.
Two different pairs trigger on the same date	Both are retained; date clustering is handled in robustness/economic evaluation, not by deleting valid events.

5.3 Stable IDs and reproducibility

The same frozen inputs and label policy must regenerate the same event IDs and outcomes. Event construction is deterministic and occurs before any model training. Changing z_entry, z_exit, z_stop, H, duplicate rules, or pair-eligibility policy changes label_policy_version and creates a new dataset version.

6. Feature engineering design principles

6.1 Point-in-time feature vector

$$X_i = (x_{i1}, x_{i2}, \ldots, x_{ip}) \subseteq \mathcal{I}_{t_i}$$

Every feature is computed from the historical information set at the event timestamp. Rolling windows end at t0, econometric parameters come from the latest frozen relationship snapshot, and any learned transformation is fitted on training data only.

6.2 Feature architecture

Family	Core v1?	Purpose
Spread state	Yes	How extreme and dynamically unstable is the current dislocation?
Relationship quality	Yes	How strong and stable is the historical pair relation?
Market context	Yes	Is the event occurring in calm, directional, or stressed conditions?
Security state	Yes	Is the divergence associated with one-sided price/volume behaviour?
Liquidity / implementation	Yes	Is the event occurring in liquid names under normal daily trading conditions?
Structured event / NLP	Model B	Adds source-grounded company-specific catalyst information after the NLP layer exists.

6.3 Feature exclusions

- No ticker or pair identity feature.
- No future return, future spread path, backtest result, or label-related field.
- No point-in-time fundamental valuation variables in v1 because reliable historical snapshots are not guaranteed by the current data architecture.
- No deep embeddings or raw text in the tabular XGBoost model. NLP is converted to a small structured schema first.

Core Model A before Model B

Build and evaluate the quantitative feature set first. Only after the NLP pipeline can reconstruct historical evidence point-in-time should structured event features be added and the same training protocol rerun as Model B. This preserves a clean ablation.

7. Core feature set: spread state

These variables describe the current residual-spread state. The event sign is used to create direction-normalised dynamics: positive aligned velocity always means the spread is moving further away from equilibrium, regardless of whether the original z-score is positive or negative.

Feature	Definition / transformation	Interpretation
z_signed	Current frozen z_t	Signed position relative to equilibrium.
abs_z	|z_t|	Magnitude of divergence.
divergence_sign	sign(z_t)	Positive versus negative spread side.
adverse_z_velocity_1d	sign(z_t) * (z_t - z_{t-1})	Positive = divergence worsening over one day.
adverse_z_velocity_5d	sign(z_t) * (z_t - z_{t-5}) / 5	Average five-day movement further from/toward equilibrium.
spread_vol_20	Std. dev. of daily spread changes over trailing 20 days	Short-run instability of the residual.
spread_vol_ratio_20_60	spread_vol_20 / spread_vol_60	Recent volatility expansion versus medium-run state.
days_since_equilibrium	Trading days since |z| last entered the <=0.5 band	Recency of normalisation.
equilibrium_crossings_60	Count of equilibrium-band entries / sign crossings in prior 60 days	Empirical reversion activity.
max_abs_z_20	Maximum |z| in trailing 20 days	Whether current event is unusually extreme even relative to recent stress.

7.1 Trailing-window convention

All windows include information through event day t0 only. If a required lookback is unavailable, the event is normally rejected as not feature-complete rather than backfilled with future values.

8. Core feature set: relationship quality

Relationship features are read from the latest frozen pair snapshot and its trailing history. They describe whether the statistical relationship looks robust before the divergence occurs.

Feature	Construction	Interpretation
coint_tstat	Engle-Granger statistic from current snapshot	More negative values generally indicate stronger evidence against no cointegration.
coint_pvalue	MacKinnon approximate p-value	Direct validity measure used by the relationship engine.
coint_strength	-log10(max(p, 1e-6))	Monotone strength transformation useful for linear baseline and diagnostics.
half_life	Estimated spread half-life, clipped only by pre-declared data-quality bounds	Expected speed of mean reversion, not a guaranteed exit time.
beta_abs	|beta_hat|	Magnitude of long-run relative exposure.
beta_drift_pct	|beta_current - beta_previous| / (|beta_previous| + eps)	Recent hedge-ratio instability.
corr_20	Trailing 20-day return correlation	Short-run co-movement.
corr_60	Trailing 60-day return correlation	Medium-run co-movement.
valid_months_last_6	Number of last 6 monthly snapshots passing relationship eligibility	Recent validity persistence.

No re-estimation at event time

The ML pipeline must not refit the pair relationship specifically because a divergence occurred. It uses the same point-in-time snapshot that the scanner would have had in production. This prevents event-conditioned look-ahead and preserves the separation between econometrics and ML.

9. Core feature set: market context state

Market features provide context because mean reversion may behave differently in broad sell-offs, sector shocks, or volatility spikes. These are observable covariates, not a separate forecasting model.

Feature	Data / transformation	Timing
market_return_1d	SPY log return over t-1 -> t	Known after event-day close.
market_return_5d	Cumulative SPY log return over trailing 5 trading days	Past-only.
sector_return_1d	Mapped sector ETF return over t-1 -> t	Past/current only.
sector_return_5d	Mapped sector ETF cumulative trailing 5-day return	Past-only.
vix_level	VIX close on t	Known at end-of-day.
vix_change_5d	VIX_t - VIX_{t-5}	Volatility shock context.
market_rv20	Annualised or unannualised std. dev. of SPY daily returns over trailing 20 days; one convention fixed globally	Realised market volatility.

10. Core feature set: security state and liquidity

10.1 Security-state features

Feature	Construction	Interpretation
aligned_relative_return_5d	sign(z_t) * (R_A,5 - R_B,5)	Positive = recent relative move reinforces current divergence.
aligned_relative_return_20d	sign(z_t) * (R_A,20 - R_B,20)	Medium-run relative momentum aligned to divergence direction.
abnormal_return_A_1d	R_A,1 - R_sector,1	One-day move of leg A beyond sector move.
abnormal_return_B_1d	R_B,1 - R_sector,1	One-day move of leg B beyond sector move.
volume_z_A_20	(Volume_A,t - mean_20) / std_20	Unusual activity in leg A.
volume_z_B_20	Same for leg B	Unusual activity in leg B.
relative_realised_vol_20	RV_A,20 - RV_B,20	Asymmetry in idiosyncratic volatility.

10.2 Liquidity / implementation features

Feature	Construction	Interpretation
log_min_adv20	log(1 + min(ADV_A,20, ADV_B,20)) where ADV = mean(price * volume)	Liquidity of the weaker leg.
adv_ratio	max(ADV_A, ADV_B) / min(ADV_A, ADV_B)	Liquidity imbalance across legs.
max_range_proxy20	max(mean((high-low)/close)_A, mean((high-low)/close)_B) over 20d	Daily-data proxy for noisier / wider trading conditions.

Why not use estimated bid-ask or borrow costs as core predictors?

The prototype does not have a defensible historical point-in-time borrow/quote dataset. Transaction costs are handled explicitly in backtesting. We should not fabricate precise microstructure features from daily public data and then treat them as production-quality.

11. Structured event / NLP features for Model B

Model B adds structured company-event features after the event-intelligence pipeline can retrieve and classify pre-event SEC evidence reproducibly. Model A is built first, then the same frozen training protocol is rerun with the event-feature family to create Model B.

Logical field	Encoding for XGBoost	Availability rule
event_type	One-hot: EARNINGS, GUIDANCE, M&A, REGULATORY, LITIGATION, MANAGEMENT, PRODUCT/OTHER, NONE/UNKNOWN	Source document must pre-date t0.
affected_leg	One-hot: A, B, BOTH, NONE/UNKNOWN	Must come from structured extraction.
materiality	One-hot LOW / MEDIUM / HIGH / UNKNOWN	No free-form LLM score.
one_sided_event	Binary	1 when only one company has a material event.
source_count	Integer count of admissible supporting documents	Only admissible pre-event sources.
document_recency_days	Days since most recent relevant admissible document	Capped by evidence lookback policy.
extraction_confidence	0-1 structured extractor confidence	Displayed as model input only if calibration/meaning is documented.
evidence_available	Binary	Explicitly distinguishes NONE from missing retrieval.

11.1 Strict ablation requirement

Headline ML evidence must compare Model A (quantitative features only) with Model B (Model A + structured event features extracted from admissible SEC evidence). This directly tests whether NLP-derived structure adds predictive information rather than hiding it inside a single opaque final system.

12. Supervised dataset schema and storage

12.1 One row per event

Field group	Examples	Storage rule
Keys / versions	event_id, pair_id, event_timestamp, dataset_version, feature_schema_version, label_policy_version	Identifiers retained but pair/ticker IDs excluded from model matrix.
Features	All feature columns defined in Sections 7-11	Immutable event feature snapshot.
Missingness / quality	feature_complete, evidence_available, quality flags	Used for validity and controlled missingness.
Target	y_revert_success	Only visible to training/evaluation jobs, never inference.
Outcome diagnostics	outcome_type, resolution_date, max adverse/favourable z	Stored beside target for analysis, excluded from X.
Lineage	snapshot_id, source dataset versions, feature_cutoff_timestamp	Enables audit back to upstream data.

12.2 Storage layout

Artefact	Recommended storage	Purpose
divergence_events	DuckDB analytics table + Parquet snapshot	Stable event registry and label metadata.
event_features	DuckDB + Parquet	Versioned point-in-time feature vectors.
supervised_dataset_<version>.parquet	Parquet	Frozen matrix used for one experiment family.
validation_predictions.parquet	Artefact directory	Calibration, threshold selection, diagnostics.
test_predictions.parquet	Artefact directory	Untouched final results after methodology freeze.

12.3 Required uniqueness and integrity rules

- event_id unique.
- Exactly one feature row per event per feature_schema_version.
- Exactly one target per event per label_policy_version.
- feature_cutoff_timestamp <= event_timestamp.
- No target column or outcome-derived field appears in the feature manifest.
- Model-ready column order fixed by feature_schema.json, never inferred ad hoc at prediction time.

13. Chronological partitions, embargo, and model-selection protocol

13.1 Fixed outer partitions

Partition	Dates	Role
Training	2018-01-01 to 2021-12-31 usable events	Fit preprocessing and model; internal forward folds for hyperparameter selection.
Validation	2022-01-01 to 2023-12-31 usable events	Fit probability calibrator; compare feature families; select policy threshold; freeze methodology.
Final test	2024-01-01 to 2025-12-31 usable events	Opened once for headline out-of-sample evaluation.
Embargo	At least H=10 trading days at each boundary	Exclude events whose future label window would cross into the next partition.

13.2 Forward-chaining model-selection folds inside training

Fold	Fit period	Evaluation period
1	2018	2019
2	2018-2019	2020
3	2018-2020	2021

Each fold applies the same event-horizon embargo. Learned preprocessing for the logistic baseline is re-fitted inside each fold. XGBoost is fit only on the fold training years. No random K-fold split is used for headline model selection.

Why this is stricter than a normal train/test split

Financial observations are ordered and market conditions change. Random splitting would allow future regimes and repeated pair behaviour to influence earlier validation rows. Chronological folds approximate the direction of real deployment: past -> future.

14. Preprocessing and training/inference data contracts

14.1 Shared feature manifest

A versioned feature manifest defines column name, type, financial meaning, source, lookback, allowed missingness, and transformation. Training and inference both call the same feature builder and validate against the same manifest.

14.2 Logistic-regression preprocessing

- Continuous numeric features: training-only median imputation if permitted by the feature contract, followed by StandardScaler fitted on the training fold.
- Binary features: 0/1 and no scaling required.
- Categorical structured-event features: one-hot encode with an explicit UNKNOWN/NONE category and unknown-category-safe inference.
- Highly skewed liquidity measures are pre-defined log transformations before scaling (for example log_min_adv20).
- No preprocessing object is fit on validation or test.

14.3 XGBoost preprocessing

- No numeric standardisation is required for tree splits.
- Pre-defined numeric transformations (direction normalisation, log liquidity, ratios) are computed by feature engineering, not learned by XGBoost.
- Missing numeric values remain NaN only when the feature contract explicitly permits them; core required features should usually cause an event-quality failure instead.
- Categorical event features are one-hot encoded before the model so the same feature order is explicit and auditable.

14.4 Data-contract checks before fit/predict

Check	Failure response
Column names/order equal feature schema	Hard fail; never silently reorder from arbitrary input.
Expected dtypes and finite values	Hard fail for required fields; controlled NaN only where allowed.
No identifiers/target columns in X	Hard fail.
All source timestamps <= event cutoff	Hard fail / leakage test failure.
Categorical values within schema or UNKNOWN mapping	Map to UNKNOWN; log warning.
Feature schema version matches model bundle	Hard fail; model and feature builder must be version-aligned.

15. Baseline model: logistic regression

15.1 Purpose

Logistic regression is the transparent benchmark. It tests whether a linear/additive probability model using the same feature information already provides useful discrimination. XGBoost must justify its additional non-linearity and complexity relative to this baseline.

$$P(y_i = 1 \mid X_i) = \sigma(\beta_0 + X_i^\top \beta), \qquad \sigma(a) = \frac{1}{1 + e^{-a}}$$

15.2 Baseline specification

Component	Working specification
Estimator	scikit-learn LogisticRegression with L2 regularisation.
Inputs	Same quantitative Model A feature family after logistic preprocessing.
Class handling	No resampling by default; optional training-only class weights if material imbalance exists.
Tuning	Small training-fold grid over regularisation strength C; no large search.
Outputs	Probability, coefficients, ROC/PR metrics, Brier score, threshold metrics.
Interpretation	Coefficients describe conditional association in the transformed feature space; not causal effects.

Baseline discipline

The logistic model is not a straw man. It receives the same point-in-time information as XGBoost. Any reported uplift should therefore reflect the value of non-linear interactions / tree structure rather than better data access.

16. Primary model: XGBoost classifier

16.1 Prediction structure

XGBoost learns an additive ensemble of regression trees. Each new tree focuses on residual prediction error from the current ensemble, allowing non-linear thresholds and interactions across spread, relationship, market, security, and liquidity states.

$$F_M(X_i) = \sum_{m=1}^{M} \eta\, f_m(X_i), \qquad \hat{p}_i^{raw} = \sigma(F_M(X_i))$$

$$\mathcal{L} = -\sum_i \left[y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i) \right] + \sum_m \Omega(f_m)$$

16.2 Fixed modelling choices

Setting	Working choice	Reason
Objective	binary:logistic	Binary mean-reversion-success probability.
Model family	XGBoost gradient-boosted trees	Strong fit for structured moderate-size data and non-linear interactions.
Scaling	None for numeric tree inputs	Tree split locations are scale-invariant.
Identifiers	Excluded	Prevents memorisation of pair/ticker identity.
Primary feature version	Model A quantitative features	Can be built before NLP system.
Enriched feature version	Model B = Model A + structured event features	Second-stage model after reliable historical event extraction.
Explainability	SHAP TreeExplainer	Local and global post-hoc feature attribution.

16.3 Why not deep learning as the primary model

The event table is moderate in size, highly structured, and dominated by engineered tabular variables. A deep neural network would increase tuning and interpretability burden without a clear data-scale advantage. This project reserves language models for unstructured evidence and uses XGBoost where tabular classical ML is the better fit.

17. Hyperparameter tuning and model selection

17.1 Search space

Hyperparameter	Candidate range	Control purpose
n_estimators	200, 300, 450, 600	Ensemble size.
max_depth	2, 3, 4, 5	Tree complexity / interaction depth.
learning_rate	0.03, 0.05, 0.08, 0.10	Shrinkage.
min_child_weight	1, 3, 5, 8	Minimum support for a child node.
subsample	0.70, 0.85, 1.00	Row stochasticity / regularisation.
colsample_bytree	0.70, 0.85, 1.00	Feature stochasticity / regularisation.
reg_alpha	0, 0.1, 0.5, 1.0	L1 regularisation.
reg_lambda	1, 3, 5, 10	L2 regularisation.
gamma	0, 0.1, 0.3	Minimum split gain.

17.2 Search method and selection rule

Use a reproducible random/parameter sample of a bounded number of configurations rather than an exhaustive high-dimensional grid. Each configuration is evaluated on the three forward-chaining training folds. The primary selection metric is mean ROC-AUC across folds, with PR-AUC, Brier score, and fold-to-fold stability used as secondary checks. The final chosen hyperparameters are then fitted once on all 2018-2021 training events.

Search budget

A course-scale target of approximately 30-50 parameter configurations is enough to demonstrate disciplined tuning without creating hundreds of researcher degrees of freedom. The search seed, candidate space, selected configuration, and all fold scores are logged in MLflow.

17.3 Feature-family decisions

Feature removal/addition is treated like a hyperparameter decision and must be made using training/validation evidence. The final test is never used to decide that a feature, window, or transformation should be changed.

18. Class imbalance, weighting, and sample dependence

18.1 Measure imbalance before correcting it

The success rate is an empirical property of the event/label policy and should be reported for each partition. Accuracy alone is not a suitable metric if one class dominates.

18.2 Default policy

- Do not use SMOTE or random over/under-sampling in the baseline pipeline. These methods complicate temporal dependence and are unnecessary unless imbalance is severe.
- Train the unweighted models first. If the minority class is materially under-represented, compare a training-only inverse-frequency/class-weight specification as a sensitivity.
- Any weight is calculated from the current training fold only; validation and test retain their natural class distribution.
- Threshold tuning is separate from class weighting. A 0.50 decision threshold is not assumed to be economically optimal.

18.3 Dependence that remains

Events from the same pair and contemporaneous market conditions are not statistically independent in the strict cross-sectional sense. The pipeline reduces the strongest dependence by suppressing overlapping same-pair episodes and using chronological evaluation. Robustness is then reported by pair, sector, year, and market-stress bucket rather than pretending every row is iid.

Do not “balance away” finance

The class frequency itself contains information about how often threshold divergences succeed. Synthetic resampling can distort that base rate and probability calibration. The app therefore prioritises natural-distribution evaluation and calibrated probabilities.

19. Probability calibration

19.1 Why calibration is a separate stage

XGBoost is trained to discriminate successful from failed events, but its raw probability estimates need not equal empirical frequencies. The UI and controller consume a probability-like score, so calibration is fitted after the base model is frozen.

19.2 Platt / sigmoid calibration

$$\hat{p}_i^{cal} = \sigma(a\, m_i + b)$$

m_i is the frozen XGBoost raw margin/logit for validation event i. Parameters a and b are fitted only on 2022-2023 validation outcomes. The resulting calibrator is then applied unchanged to 2024-2025 test margins and to app inference.

19.3 Why sigmoid is the default

- Requires only two calibration parameters and is therefore more stable for a moderate event sample.
- Keeps the mapping monotonic: better raw scores cannot be reordered by calibration.
- Isotonic regression may be evaluated as a sensitivity only if the validation sample is large enough; it is not the core choice.

19.4 Calibration outputs

Output	Use
Brier score	Primary probability-quality metric.
Reliability / calibration curve	Predicted probability bins versus observed reversion rate.
Calibration intercept/slope or validation mapping	Audit how raw XGBoost probabilities were transformed.
Raw + calibrated probability stored together	Separates model ranking from calibration layer.

20. Model thresholding and probability buckets

20.1 Probability is model output; threshold is policy

The classifier produces a continuous calibrated probability. The value used to call an event “ML-qualified” is not part of XGBoost itself; it is a downstream policy parameter selected on validation data and versioned separately.

Policy element	Working specification
Candidate threshold	0.65 starting value; validation-tuned and then frozen.
Sensitivity grid	For example 0.55 / 0.60 / 0.65 / 0.70 / 0.75, evaluated only on validation before test.
Selection criterion	Balance predictive precision, candidate count, calibration, and downstream economic validation; document the chosen rule before test.
Test discipline	No threshold change after final-test outcomes are observed.

20.2 UI probability buckets

For visual grouping, LOW / MEDIUM / HIGH model-confidence cut points should be derived from the validation probability distribution or fixed policy values and stored in the policy configuration. They are presentation/decision aids, not additional learned classes.

Do not optimise a “magic threshold” on the test set

A high Sharpe at one threshold is not evidence if that threshold was chosen after inspecting the test backtest. The test period should receive exactly the frozen model, calibrator, and policy configuration.

21. SHAP explainability

21.1 Local additive explanation

$$F(X_i) = \phi_0 + \sum_{j=1}^{p} \phi_{ij}$$

For XGBoost, TreeExplainer decomposes the base model score into a baseline plus feature contributions. Positive SHAP values push the raw model score toward predicted success; negative values push it toward failure. The calibrated probability is displayed separately because the calibrator is a post-model mapping.

21.2 Outputs saved for each event

SHAP artefact	App / analysis use
Top positive drivers	Pair Detail: factors supporting mean reversion.
Top negative drivers	Pair Detail: factors reducing expected reversion.
Full SHAP vector	Audit / model diagnostics.
Global mean absolute SHAP	Feature-importance summary across validation/test.
Dependence plots for selected features	Research diagnostics, not required in primary UI.
SHAP background/reference sample	Versioned artefact enabling reproducible explanations.

21.3 Interpretation limits

- SHAP explains the fitted model, not causal finance mechanisms.
- Correlated features can share or redistribute attribution.
- A strong SHAP contribution does not prove that changing the feature would change the outcome.
- The UI should label explanations as model drivers, not causal reasons for the divergence.

22. Predictive evaluation and calibration diagnostics

22.1 Discrimination metrics

Metric	What it tests	Headline?
ROC-AUC	Ranking of successful events above failures across thresholds.	Yes
PR-AUC	Precision-recall performance; especially useful if class balance is uneven.	Yes
Precision	Share of selected predicted successes that actually revert.	At policy threshold
Recall	Share of actual successes captured by the threshold.	At policy threshold
F1	Harmonic mean of precision and recall.	Secondary
Confusion matrix	Counts of TP/FP/TN/FN at frozen threshold.	Yes

22.2 Probability-quality metrics

$$\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}(\hat{p}_i - y_i)^2$$

Report Brier score before and after calibration, together with a calibration curve. Lower Brier indicates better probabilistic accuracy. Probability buckets should show monotonically improving empirical success rates if the model ranking and calibration are useful.

22.3 Evaluation tables required

- Logistic baseline versus raw XGBoost versus calibrated XGBoost.
- Training-fold mean/std metrics, validation metrics, and final-test metrics clearly separated.
- Metrics by positive versus negative divergence.
- Metrics by pair, sector, calendar year, and market-stress slice.
- Observed success rate by calibrated probability quintile or decile.

23. ML ablations, robustness, and finance handoff

23.1 Required ablation ladder

Version	Features / model	Question
L0	Logistic regression on core quantitative features	Is a simple additive model sufficient?
A0	XGBoost: spread + relationship only	Does non-linearity improve the pure pair signal?
A1	A0 + market context	Does broader market context add information?
A2	A1 + security + liquidity	Do one-sided security state and implementation context add information?
B	A2 + structured event/NLP features	Does source-grounded event structure add incremental predictive value?

23.2 Finance handoff

The supervised ML pipeline does not claim economic value from AUC alone. It writes event predictions to the analytics store. The backtester then compares threshold-only, econometric-valid, ML-filtered, and later agent-controlled strategies under the same cost assumptions.

ML output passed downstream	Backtest / controller use
calibrated_probability	Filter/score candidate events.
raw_score	Audit and calibration diagnostics.
probability_bucket	Selection-efficiency tables and UI grouping.
model_version / feature version	Reproducibility.
top SHAP drivers	Human-readable model diagnostics.
missingness / quality flags	Risk gate or ABSTAIN path.
prediction_timestamp	Proof that prediction uses a frozen event state.

Separation of prediction and trading evaluation

The classifier is optimised and validated as a predictive model first. Trading metrics are an external economic evaluation of using those predictions. This avoids silently training the model directly to maximise one backtest realisation.

24. Artefacts, versioning, experiment tracking, and persistence

24.1 Model bundle

Artefact	Recommended filename / format
XGBoost model	xgb_model.json or native XGBoost format
Logistic baseline pipeline	logistic_pipeline.joblib
Probability calibrator	platt_calibrator.joblib
Feature manifest	feature_schema.json
Label policy	label_policy.json
Split definition	split_policy.json
Threshold / bucket policy	ml_policy.json
SHAP reference sample	shap_background.parquet
Validation predictions	validation_predictions.parquet
Final test predictions	test_predictions.parquet
Metrics / model card	metrics.json + model_card.md/json

24.2 Version identifiers attached to every prediction

- dataset_version
- label_policy_version
- feature_schema_version
- split_version
- model_version
- calibrator_version
- ml_policy_version
- code_commit / Git SHA

24.3 MLflow experiment record

Each run logs data/version IDs, candidate hyperparameters, fold metrics, chosen feature families, class balance, calibration method, threshold policy, artefact paths, and final evaluation results. The app displays the model version used for each event.

25. Inference workflow, UI integration, and failure handling

25.1 Current/historical event inference

Stage	Action
1. Validate event	Confirm ACTIVE pair, frozen snapshot, event timestamp, and feature schema.
2. Build features	Construct X_t from data available through event close only.
3. Validate schema	Types/order/missingness/timestamps checked against feature manifest.
4. Base inference	XGBoost returns raw margin and raw probability.
5. Calibration	Saved Platt calibrator maps raw margin to calibrated P(revert).
6. Explain	Tree SHAP calculated using the exact base model/version.
7. Persist	model_predictions row written with versions and top drivers.
8. UI / controller	Scanner and Pair Detail read prediction; controller later combines it with agent/risk inputs.

25.2 Model prediction record

Field	Example content
event_id	Stable event key
model_version	xgb_model_A_v003
raw_margin	Base XGBoost score
raw_probability	Uncalibrated sigmoid probability
calibrated_probability	Displayed P(revert)
probability_bucket	LOW / MEDIUM / HIGH according to policy
top_shap_positive / negative	Feature names + values/contributions
feature_schema_version	Exact expected input schema
prediction_timestamp	Inference audit timestamp

25.3 Failure handling

- Schema/version mismatch -> no prediction; surface data/model configuration error.
- Missing required quantitative feature -> no prediction; event routed to data-quality REVIEW/ABSTAIN downstream.
- Calibrator missing or wrong version -> do not display calibrated probability.
- SHAP failure -> prediction may remain valid, but explanation status is marked unavailable; never fabricate drivers.
- Model file cannot load -> use cached historical predictions for demo mode and clearly mark inference unavailable.

26. Automated tests, monitoring, build sequence, and acceptance criteria

26.1 Required automated tests

Test family	Minimum checks
Event tests	Threshold crossing creates exactly one event; no overlapping same-pair event; deterministic event ID.
Label tests	Known synthetic z paths produce CONVERGED / STOPPED / TIMEOUT correctly; no t0 information enters target incorrectly.
Leakage tests	All feature source timestamps <= t0; no future snapshot; no outcome field in X; partition embargo enforced.
Feature tests	Trailing windows match fixtures; direction-normalised signs correct; train/inference feature vectors identical.
Split tests	Chronological partitions fixed; final test rows never appear in tuning/calibration.
Model tests	Logistic and XGBoost fit/load; probabilities in [0,1]; deterministic inference for fixed artefact.
Calibration tests	Calibrator fit only on validation; saved mapping reproduces probabilities.
SHAP tests	Expected number/order of contributions; additive reconstruction within tolerance.
Persistence tests	Prediction row contains all required version IDs and links to event/feature row.

26.2 Build sequence

Order	Deliverable	Acceptance condition
1	Event generator + label resolver	Frozen historical events reproduce exactly on rerun.
2	Feature builder + schema	All core Model A features constructed point-in-time with no leakage failures.
3	Supervised dataset + splits	Train/validation/test and embargo counts documented.
4	Logistic baseline	Complete baseline metrics and saved pipeline.
5	XGBoost tuning + final training model	Forward-fold search reproducible; selected model saved.
6	Validation calibration + threshold policy	Calibrator and policy frozen before test.
7	Final test + SHAP	One untouched evaluation with full diagnostic tables.
8	Inference integration	Pair Detail can load an event and reproduce its stored probability/SHAP explanation.
9	Model B event-feature extension	Structured SEC event features added after event intelligence is defensible; same protocol rerun.

26.3 Minimum acceptance criteria

- No automated point-in-time/leakage test failures.
- XGBoost must be compared against logistic baseline, not presented in isolation.
- Calibration curve and Brier score reported in addition to ranking metrics.
- Every displayed probability is tied to event, feature, model, calibrator, and policy versions.

- Final-test results are generated only after method freeze; any later method change creates a new experiment and invalidates the old "final" test claim.

Appendix A. Working configuration defaults

Parameter	Working default	Governance
Event trigger	|z| >= 2.0 on fresh crossing	Label-policy versioned; validation sensitivity allowed.
Convergence band	|z| <= 0.5	Target definition.
Adverse stop	Same-side |z| >= 3.0	Target definition.
Label horizon	10 trading days	Target definition and embargo minimum.
Training	2018-2021	Model fit + forward folds.
Validation	2022-2023	Calibration, feature/threshold decisions.
Final test	2024-2025	Untouched until freeze.
Baseline	L2 logistic regression	Same feature information.
Primary classifier	XGBoost binary:logistic	Model A first; Model B event-feature extension.
Hyperparameter search	~30-50 bounded configurations, 3 forward folds	Logged and reproducible.
Calibration	Sigmoid / Platt on validation predictions	Saved separately from base model.
Candidate threshold	0.65 starting value	Validation-tuned policy, then frozen.
Class handling	Natural distribution; no resampling by default	Class weights only as training sensitivity.
Explainability	Tree SHAP on base XGBoost model	Explain raw model score; display calibrated probability separately.
Core storage	DuckDB + Parquet + MLflow artefacts	Versioned event/feature/prediction lineage.

Defaults are not truths

The numeric label and threshold values define one coherent research experiment. If validation evidence motivates a change, update the relevant policy version before opening the final test. Do not tune these numbers retrospectively against 2024-2025 performance.

Appendix B. Formula and feature reference

B.1 Core formulas

$$|z_{t_0}| \geq z_{entry}, \qquad |z_{t_0-1}| < z_{entry}$$

$$y_i = 1 \;\Leftrightarrow\; \tau_i^{conv} \leq H \text{ and } \tau_i^{conv} < \tau_i^{stop}$$

$$P(y = 1|X) = \sigma(\beta_0 + X^\top \beta)$$

$$F_M(X) = \sum_{m=1}^{M} \eta f_m(X), \qquad \hat{p} = \sigma(F_M(X))$$

$$\hat{p}^{cal} = \sigma(a\, m + b)$$

$$\text{Brier} = N^{-1} \sum_{i=1}^{N} (\hat{p}_i - y_i)^2$$

$$F(X) = \phi_0 + \sum_{j=1}^{p} \phi_j$$

B.2 Feature count by family - core v1

Family	Candidate count	Notes
Spread state	10	Current extremeness, velocity, residual volatility, crossing history.
Relationship quality	9	Cointegration strength, half-life, beta stability, correlations, validity history.
Market context	7	Market/sector returns, VIX, realised volatility.
Security state	7	Relative momentum, abnormal returns/volume, relative volatility.
Liquidity / implementation	3	Minimum ADV, imbalance, daily range proxy.
Core Model A total	36	Exact feature manifest may remove a feature using training/validation evidence only.
Structured event/NLP	~8 logical fields before one-hot expansion	Model B feature family, added after the Model A pipeline is validated.

B.3 Fields intentionally excluded from the model matrix

- event_id, pair_id, ticker/company identifiers;
- outcome_type, resolution date, exit information, future z-path, trade return;
- future or revised econometric snapshots;
- post-event filings/news;
- final agent verdicts and final controller decision;
- any value calculated using the final test period during training/selection.

References and course-source basis

This specification operationalises the architecture already defined in the project materials. It does not replace the broader system design; it narrows Sections 6-8 into a build-ready supervised-learning pipeline.

Source	Material used in this document
Project Idea 2 - AI Structural-Break Detector for Statistical Arbitrage	Event-based target, XGBoost/LightGBM rationale, SHAP, chronological evaluation, feature families, structural-break framing.
AI Structural-Break Detector - Complete Project Overview	Working scope, dates, label defaults, Model A/Model B architecture, storage, technical stack, evaluation ladder, model/version requirements.
Econometric Relationship Engine specification	Point-in-time pair snapshot, frozen spread/z-score, relationship diagnostics, event eligibility inputs.
Data Architecture & Source Strategy specification	Market/sector/VIX/OHLCV sources, timestamp discipline, DuckDB/Parquet storage, data lineage.
FINS5557 Week 1-2 materials	Classical ML taxonomy, leakage risk, XGBoost, explainability and finance deployment framing.
FINS5557 Week 5-6 materials	Model drift/governance, pipeline reproducibility, AUC/PR/confusion-matrix/SHAP workflow, Streamlit integration.
FINS5557 Week 8 materials	Short-term quantitative trading context and non-stationarity framing.

Next layer after this document

Once the quantitative Model A pipeline works end-to-end, the next major build is the point-in-time NLP / event-intelligence layer. That layer can then feed structured event features into Model B and later support the adversarial structural-break agents.